

DAY 02

Services & Networking Fundamentals

How workloads discover, reach, expose, and protect each other on k3s.

CLUSTERIP

DNS

NODEPORT

NETWORKPOLICY



What you will be able to do

Explain why Pod IP addresses are not stable application endpoints.

Choose between ClusterIP, NodePort, LoadBalancer, ExternalName, and headless Services.

Use Kubernetes DNS for service discovery and validate Pod-to-Pod traffic.

Apply a NetworkPolicy allow-list and troubleshoot failed requests.

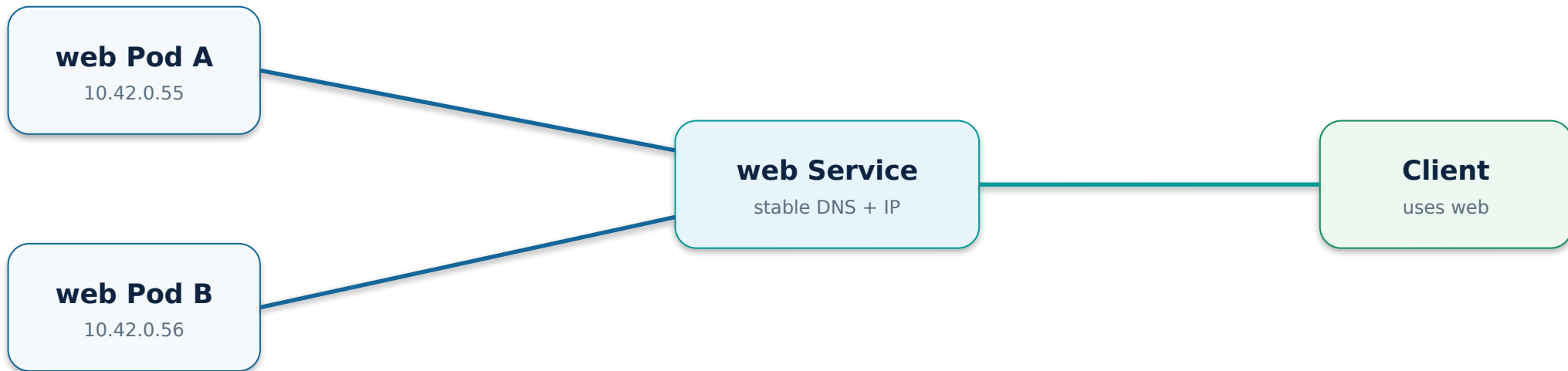
Hands-on path

1. Internal Service
2. DNS and endpoints
3. External exposure
4. Network controls

Every concept maps to a tested manifest under Day-02-Services-Networking.

The problem a Service solves

Pods are intentionally disposable. A new Pod normally receives a new IP address.



The contract

Clients call the Service name. Kubernetes tracks ready backend Pods in EndpointSlices.

Service anatomy

```
spec:  
  type: ClusterIP  
  selector: { app: web }  
  ports:  
    - port: 80  
      targetPort: 80
```

selector

Matches backend Pod labels. No matching ready Pods means no usable endpoints.

port → targetPort

The client-facing Service port forwards to the container port.

type

Controls how the Service is exposed. ClusterIP is the default.

EndpointSlice

Kubernetes records the ready, selected Pod endpoints here.

```
kubectl get svc,endpointlice -n day02-networking
```

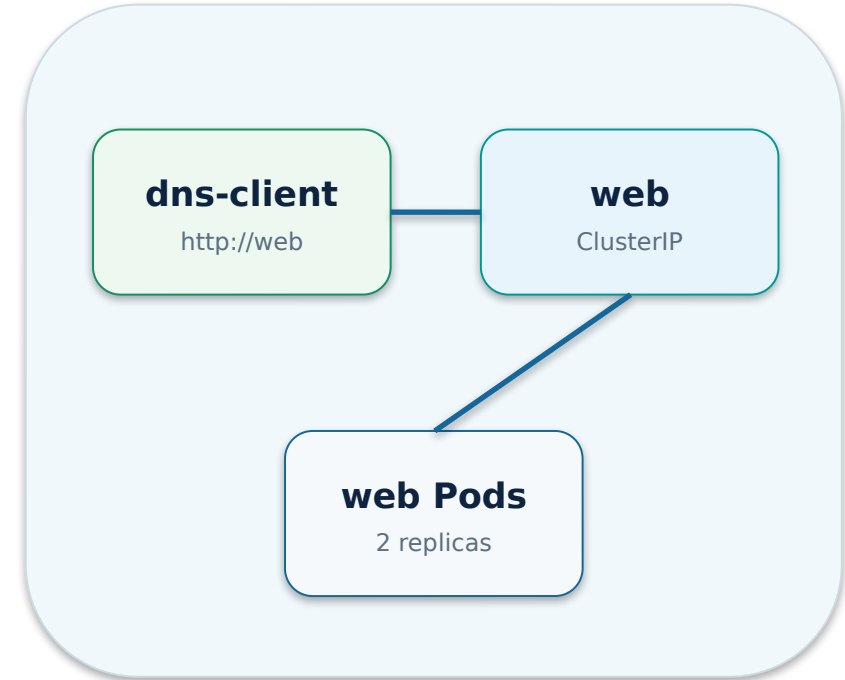
ClusterIP — the internal default

Provides one stable virtual IP reachable inside the cluster.

Best default for internal APIs, workers, databases, and microservices.

Clients use the Service name; traffic reaches a ready selected endpoint.

A ClusterIP is not a network firewall boundary.



```
kubectl exec -n $NS dns-client -- wget -qO- http://web
```

DNS is the preferred discovery mechanism

Kubernetes DNS makes the Service name portable across Pod replacements.

Same namespace

web

Cross namespace

web.day02-
networking

Fully qualified

web.day02-networking.svc.cluster.local

```
kubectl exec -n $NS dns-client -- nslookup web.day02-networking.svc.cluster.local
```

DNS updates with Service and EndpointSlice state. Environment variables are injected only when a Pod starts.

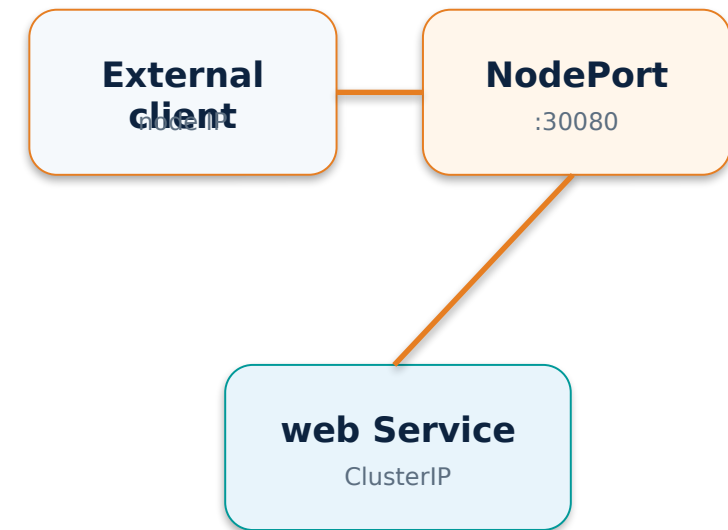
NodePort — expose through every node

Retains the underlying ClusterIP and opens one TCP port on every node.

The lab pins nodePort: 30080; reach it at <node-ip>:30080.

Useful for labs and simple on-premise testing.

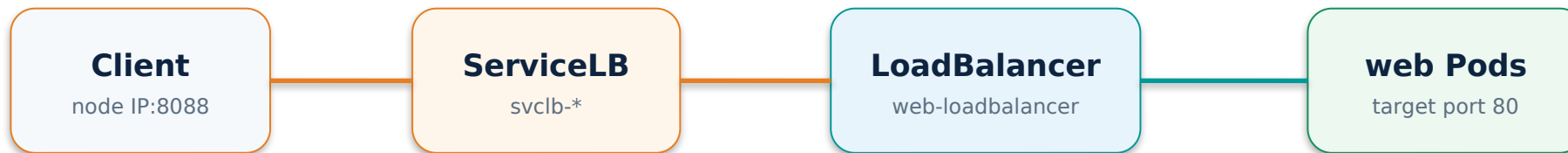
It does not provide HTTP routing, certificates, or application authentication.



```
curl -s http://$NODE_IP:30080/
```

LoadBalancer on k3s

Cloud Kubernetes provisions a cloud load balancer. Typical k3s uses ServiceLB instead.



Lab result

ServiceLB assigned 192.168.230.103. This is node-level exposure, not a managed cloud external load balancer.

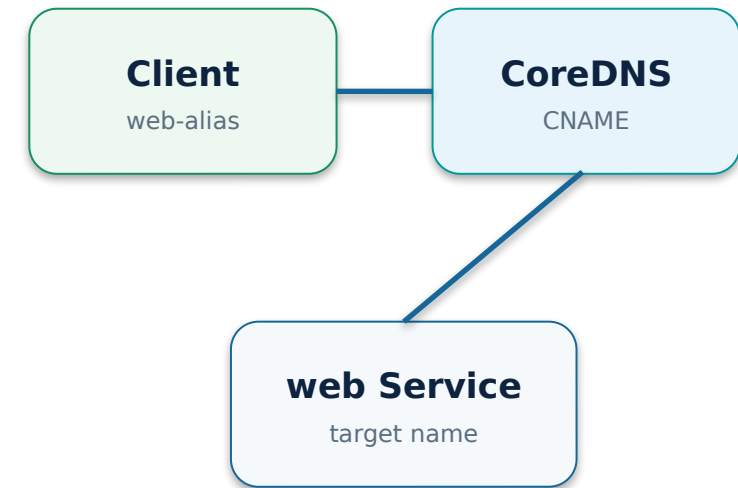
ExternalName — DNS alias only

Returns a DNS CNAME for spec.externalName.

Has no selector, ClusterIP, EndpointSlice, or kube-proxy forwarding rule.

Useful for a SaaS/database hostname alias or a staged migration.

Check TLS hostname expectations and CNAME support in the client.



```
externalName: web.day02-networking.svc.cluster.local
```

Headless Service — endpoint discovery

Set clusterIP: None when clients need individual endpoint addresses rather than a virtual IP.

```
ClusterIP DNS:  web          → 10.43.88.168 (virtual Service IP)
Headless DNS:  web-headless → 10.42.0.58, 10.42.0.59 (Pod IPs)
```

Good fit

StatefulSet peer discovery, database clusters, client-side load balancing.

Important

The client selects and connects to an endpoint directly; there is no Service virtual IP.

```
kubectl exec -n $NS dns-client -- nslookup web-headless.$NS.svc.cluster.local
```

Pod-to-Pod communication: two paths

Both routes are possible; application code normally chooses the Service DNS route.

Direct Pod IP

client → 10.42.0.56
Useful only when a client deliberately needs a specific endpoint. IP changes on replacement.

Service DNS (preferred)

client → http://web → ready endpoint
Stable name; Kubernetes tracks changing endpoints.

```
kubectl get endpointslice -n $NS -l kubernetes.io/service-name=web
```

A Service is discovery + endpoint selection. It does not itself authorize the traffic.

NetworkPolicy mental model

1. Select targets

A policy affects only Pods matched by its podSelector.

2. Isolate direction

policyTypes: Ingress and/or Egress turns the matching direction into an allow-list.

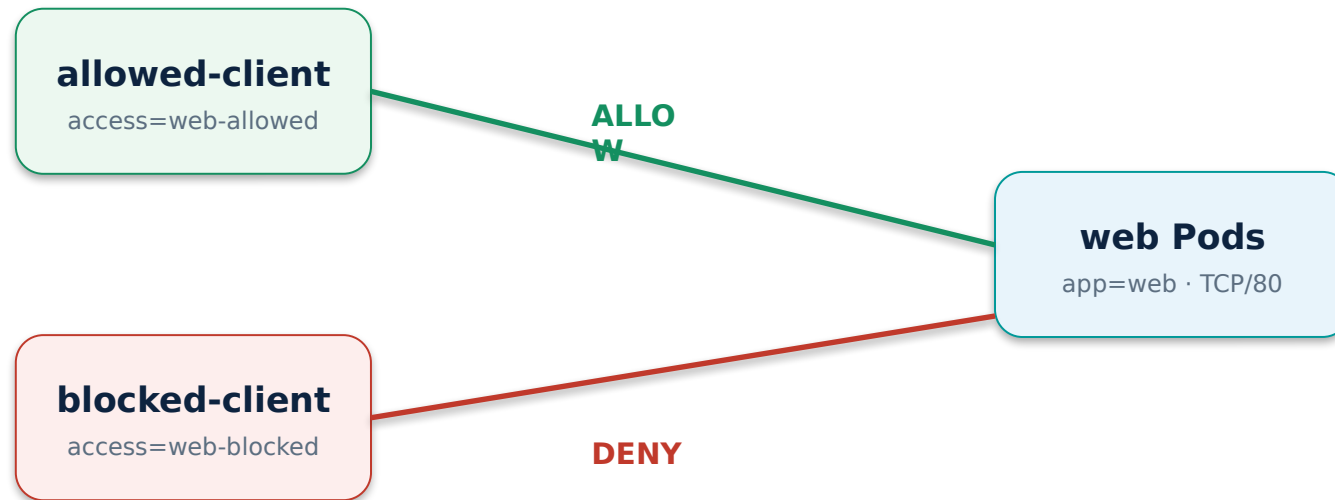
3. Add permits

Matching policies are additive: a request is allowed when at least one rule permits it.

Enforcement matters

A NetworkPolicy object alone does nothing if the selected CNI / policy controller does not enforce it. Verify this in every cluster.

NetworkPolicy lab: allow one client label



```
podSelector: { app: web }
from: podSelector:
  { access: web-allowed }
  ports: [ TCP/80 ]
```

Verified on the k3s lab: allowed client succeeded; blocked client was refused.

Troubleshooting: follow the request

1 Pod

Ready? Correct labels?

2 Service

Selector matches?

3 Endpoints

EndpointSlices populated?

4 DNS

Does the client resolve the name?

5 Policy

Source, destination, port allowed?

6 Exposure

Node firewall / ServiceLB healthy?

```
kubectl get svc,endpointlice -n $NS      # selector / ready endpoints
kubectl exec -n $NS dns-client -- nslookup web.$NS.svc.cluster.local
                                kubectl get events -n $NS --sort-by=.lastTimestamp | tail -20
```

Choose deliberately. Verify in the real cluster.

Internal API

ClusterIP + DNS

Direct members

Headless Service

Lab exposure

NodePort / ServiceLB

Traffic boundary

NetworkPolicy

Next: run the six labs in order

Apply the namespace and ex01 workload.

Observe DNS and EndpointSlices before exposing anything.

Test NodePort and k3s ServiceLB from the node.

Finish by proving the NetworkPolicy allow-list.

```
Day-02-Services-Networking/LAB-  
MANUAL.md  
Day-02-Services-Networking/  
OUTPUT.md
```

Questions?